Programming II

Packages and Class Visibility

Michele Pasqua, PhD

UNIVERSITÀ
di VERONA
Dipartimento
di INFORMATICA

A.Y. 2025/2026

Packages

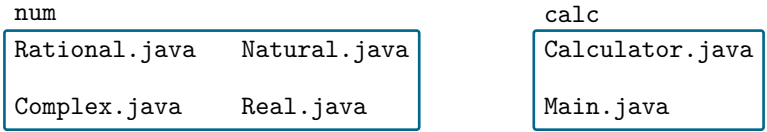
More modularity

A package is a group of classes, logically correlated

- Encapsulation adds modularity to our code, with packages we scale up

Think of a package as a library, whose interface is the set of its classes

- Technically, a package is a directory of `.java` files



More modularity (cont'd)

Packages help in making the code more structured, especially in big projects

A package defines a new namespace and scope

- Visibility is bounded by packages
- The same class name can be used under different packages

A package is identified by a hierarchical name (fully qualified name)

- For instance, `java.lang`

Naming convention: Internet name in reverse order

- For instance, `it.univr.mypackage`

Package

Packages can be nested

`java.awt.event` is a sub-package of `java.awt`

To create a package, add `package pkg;` at the beginning of the `.java` file

- If nested, make sure to create the directory tree

To use a package, use the `import` directive in the desired `.java` file

- `import pkg.MyClass;` will import a single class (`MyClass`) from `pkg`
- `import pkg.*;` will import all classes of `pkg` (without sub-packages)

Default package

When no package is specified, then a class belongs to the default package

- The default package has no name
- Classes in the default package cannot be accessed by classes of other packages

⚠ Usage of the default package is a bad practice and is discouraged

Directory tree

Compiler viewpoint (source)

```
src/        not a package
  num/
    Natural.java
    Rational.java
    Real.java
    Complex.java
  calc/
    Calculator.java
    Main.java
```

Interpreter viewpoint (bytecode)

```
bin/        not a package
  num/
    Natural.class
    Rational.class
    Real.class
    Complex.class
  calc/
    Calculator.class
    Main.class
```

Deployment with packages

```java
package pkg;                                    // source file:
public class MyClass {                          // src/pkg/MyClass.java
  public static void main(String[] args) {
    System.out.println("Hello World!");
  }
}
```

Compile with `javac -d bin/ src/pkg/MyClass.java`

- This will generate `bin/pkg/MyClass.class`

Package with `jar cvf MyJar.jar -C bin/ pkg/`

Run with `java -cp MyJar.jar pkg.MyClass`

Deployment with packages (cont'd)

To define a main class, add a manifest file

```
jar cvfm MyJar.jar manifest.txt -C bin/ pkg/
```

where `manifest.txt` contains the line `Main-Class: pkg.MyClass`

Then, run with `java -jar MyJar.jar`

- In large projects, let build tools (e.g., Maven) to take care of packaging

Automatic building and deployment

By using a build tool like Maven with a suitable plugin

```
<build>             <!-- to add to the pom.xml Maven configuration file -->
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-assembly-plugin</artifactId>
      <version>3.7.1</version>
      <configuration>
        <descriptorRefs>
          <descriptorRef>jar-with-dependencies</descriptorRef>
        </descriptorRefs>
        <archive>
          <manifest>
            <addClasspath>true</addClasspath>
            <mainClass>pkg.MainClazz</mainClass>
          </manifest>
        </archive>
      </configuration>
...
```

Automatic building and deployment (cont'd)

By using a build tool like Maven with a suitable plugin

```
...
      <executions>
        <execution>
          <id>assemble-all</id>
          <phase>package</phase>
          <goals>
            <goal>single</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Build with `mvn package`

- A `projectName-1.0-SNAPSHOT-jar-with-dependencies.jar` will be created

Class Visibility

Package resolution

Each class is local to the package where it is defined

- To use a class from another package one needs to tell Java where to find it

With fully qualified names

```
package calc;

class Calculator {
  public static num.Real div(num.Natural a, num.Natural b) {
    // compute the division of a by b
  }
}
```

Package resolution

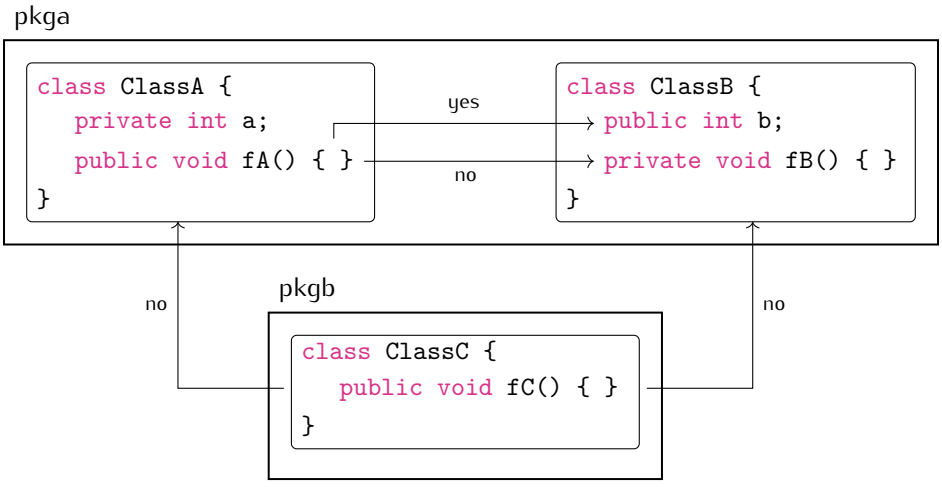
Each class is local to the package where it is defined

- To use a class from another package one needs to tell Java where to find it

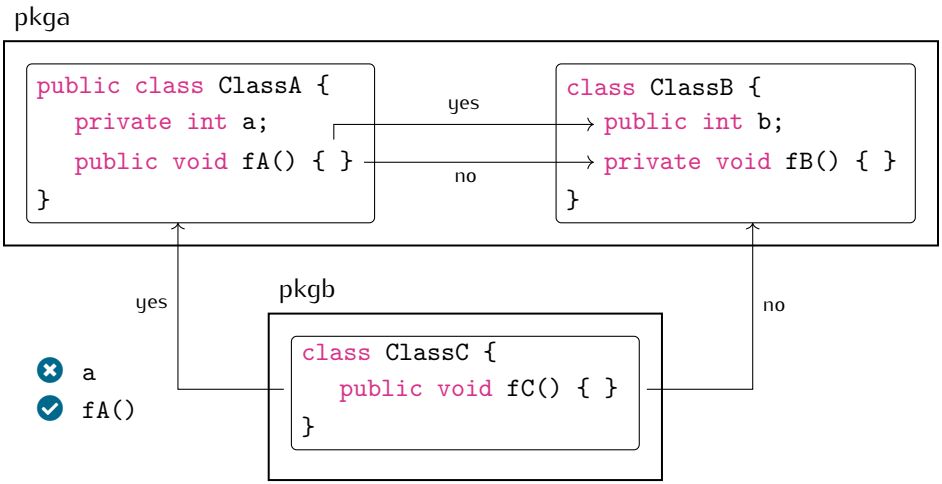
By using `import` directives

```
package calc;

import num.Real;    // this two imports can be replaced
import num.Natural; // by a single num.* import

class Calculator {
  public static Real div(Natural a, Natural b) {
    // compute the division of a by b
  }
}
```

Class visibility



Class visibility



Default class access

When omitted, the default class access is package-private

- The class is visible only by classes from the same package

Classes cannot be defined as `private`, generally

- The class and its members would be visible to themselves only

Access rules

Access rules for class members (fields and methods)

	same class	same package	other package
`private` member in any class	✓	✗	✗
`public` member in (package) class	✓	✓	✗
`public` member in `public` class	✓	✓	✓

...things will get more complicated shortly

Q + A